COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	September 14, 2026	**DATE SUBMITTED:**	September 15, 2026
NAME:	Carl Andrei D. Torres	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 **Hands-On Activity 8.1 Sorting Algorithms Pt 2**

1. PROCEDURE OUTPUT

ILO A

Code + Console Screenshot

```cpp
#include <iostream>
#include <random>
#include "sortingAlgo2.h"

const int size = 100;

int main() {
    int dataset[size];

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 100);

    for (int i = 0; i < size; i++) {
        dataset[i] = dis(gen);
    }

    std::cout << "\n----Original array----\n";
    displayArray(dataset, size);
```

```cpp
#ifndef SORTINGALGO2_H
#define SORTINGALGO2_H
#include <iostream>

template <typename T>
void displayArray(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

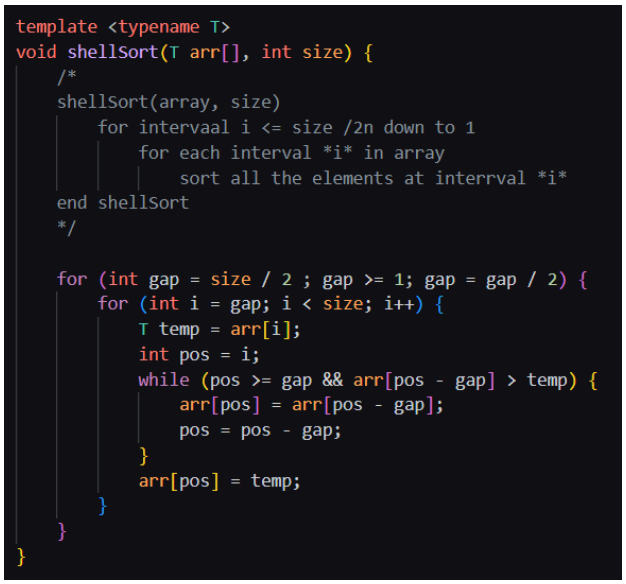
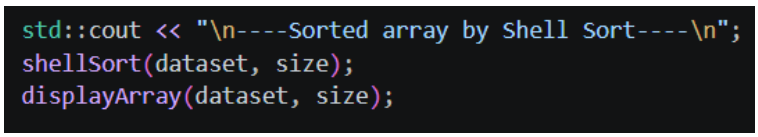
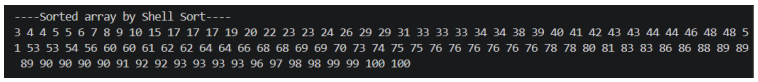
```
----Original array----
83 93 99 89 81 86 100 83 51 100 89 76 7 26 93 41 68 4 69 10 53 70 62 31 62 97 98 96 90 46 80 34 66 91 93 34 42 78 1
7 64 76 5 99 64 93 20 17 86 43 38 90 89 33 23 69 3 90 43 56 44 68 9 15 33 90 98 39 76 29 23 60 76 60 92 78 75 5 40
44 24 76 53 61 48 6 92 4 75 19 76 22 74 88 54 33 48 8 17 73 29
```

Observations

In this code and the output below, the **latest random number generators in C++ are utilized through the header file, which implements std::random_device, and std::mt19937 seed engine with std::uniform_int_distribution dis(1, 100) to directly generate integers between 1 and 100.** The **constant "size" is set to 100 in**

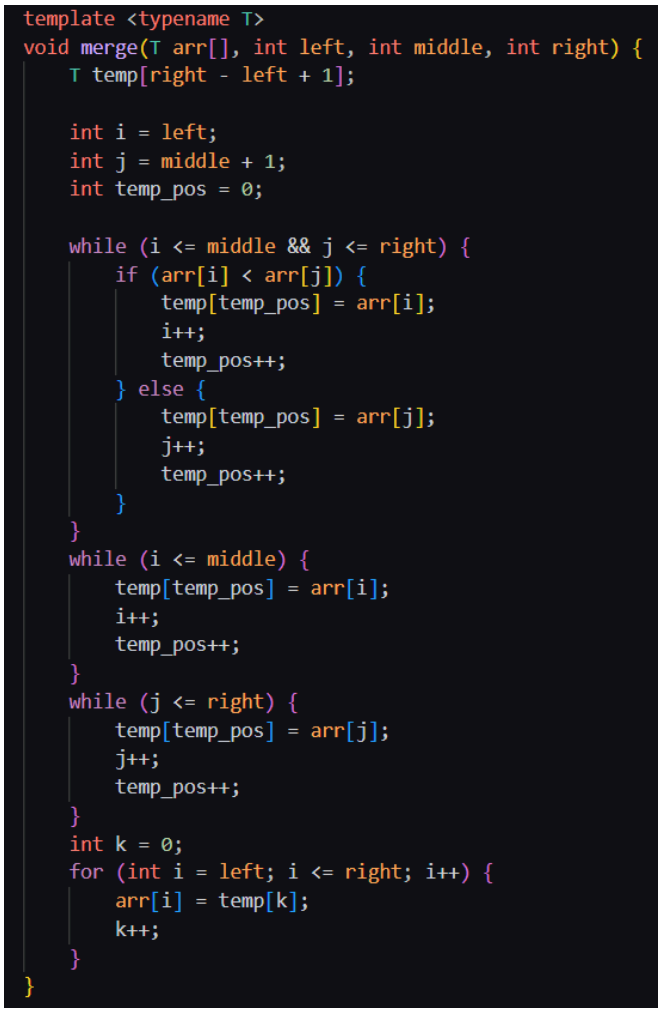
	order to define the maximum size of the integer array dataset[size], which gets filled during the body of the for loop. In addition, a **generic function to display the array values is created in the header file "sortingAlgo2.h" using C++ templates (template), which can go through all the elements of the unsorted array and display them one after another**. This function is **called in main(), which outputs "—Original array—" and the list of the 100 randomly generated integers separated by a space**.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	  
Observations	In the given code, I have **applied the algorithm of the Shell Sort**. In this algorithm, firstly, the **outer loop starts from size/2 and halves (gap=gap/2) until it reaches 1**. Then, **the middle loop runs from i=gap to size-1 and insertion sort is applied to sort array elements with a specific gap interval**. The **inner while loop compares temp variable with arr[pos -gap], and if arr[pos - gap] > temp, then it shifts the element to the right by arr[pos] = arr[pos - gap] and decreases pos by gap to continue comparing with previous elements**. The **Shell Sort algorithm has an time complexity of basic O(n^2) sorting algorithm because it reduces the distance between unsorted elements**. After the shell sort

	function call, the **given dataset will be sorted and displayed with a header "----Sorted array by Shell Sort----", where array elements are separated by a tab and represent the non-decreasing numerical order**.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```
template <typename T>
void merge(T arr[], int left, int middle, int right) {
    T temp[right - left + 1];

    int i = left;
    int j = middle + 1;
    int temp_pos = 0;

    while (i <= middle && j <= right) {
        if (arr[i] < arr[j]) {
            temp[temp_pos] = arr[i];
            i++;
            temp_pos++;
        } else {
            temp[temp_pos] = arr[j];
            j++;
            temp_pos++;
        }
    }
    while (i <= middle) {
        temp[temp_pos] = arr[i];
        i++;
        temp_pos++;
    }
    while (j <= right) {
        temp[temp_pos] = arr[j];
        j++;
        temp_pos++;
    }
    int k = 0;
    for (int i = left; i <= right; i++) {
        arr[i] = temp[k];
        k++;
    }
}
```

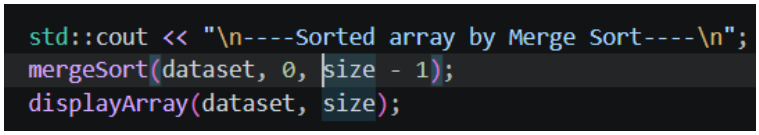
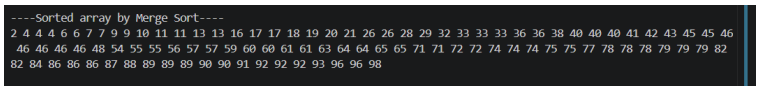
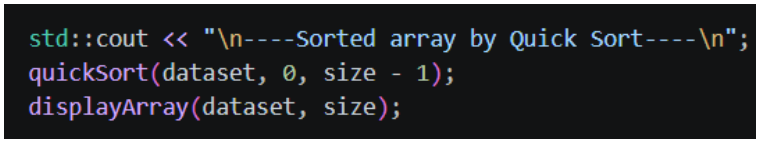
	  
Observations	In this code, I implemented the **Merge Sort algorithm that uses a divide-and-conquer strategy**, in which my recursive **mergeSort() function** keep splitting an array at **int middle = (left + right) / 2 until it reaches base cases of left >= right and sorting the left and right sub-arrays**. With **mergeSort(dataset, 0, size - 1)** called in my **main() function**, I have a **helper function merge()** that **combines them by using a temporary array temp[right - left + 1] and a primary while loop that compares the values of my left sub-array at i and my right sub-array at j**, and **two cleanup loops that grab any elements remaining**. Finally, **I finish it up with a for loop that puts them back into my original array's indices from left to right**. Its **guaranteed time complexity is O(n log n)** because **executing this code to my main() successfully, would sort my array and print out the header "----Sorted array by Merge Sort----"** and **all 100 values displayed in non-decreasing numerical order**.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

	``cpp
	template <typename T>
	int partition(T arr[], int low, int high) {
	    T pivot = arr[high]; // pivot
	    int i = (low - 1); // Index of smaller element
	
	    for (int j = low; j <= high - 1; j++) {
	        // If current element is smaller than or equal to pivot
	        if (arr[j] <= pivot) {
	            i++; // increment index of smaller element
	            std::swap(arr[i], arr[j]);
	        }
	    }
	    std::swap(arr[i + 1], arr[high]);
	    return (i + 1);
	}
	
	template <typename T>
	void quickSort(T arr[], int low, int high) {
	
	    if (low < high) {
	        // pi is partitioning index, arr[pi] is now at right place
	        int pi = partition(arr, low, high);
	
	        // Recursively sort elements before and after partition
	        quickSort(arr, low, pi - 1);
	        quickSort(arr, pi + 1, high);
	    }
	}
	``
	``
	----Sorted array by Quick Sort----
	2 4 4 4 6 6 7 7 9 9 10 11 11 13 13 16 17 17 18 19 20 21 26 26 28 29 32 33 33 33 36 36 38 40 40 40 41 42 43 45 45 46
	46 46 46 46 48 54 55 55 56 57 57 59 60 60 61 61 63 64 64 65 65 71 71 72 72 74 74 74 75 75 77 78 78 78 79 79 79 82
	82 84 86 86 86 87 88 89 89 89 90 90 91 92 92 92 93 96 96 98
	``

Observations	In this code, I have **implemented Quick Sort Algorithm using a some kind of Partitioning strategy**, namely the **recursive quickSort() function is executed as long as low < high to sort the input array**. Moreover, the **partition() function sets the last element as a pivot (pivot = arr[high]) to be compared with each element of the array, and, if they are found smaller than the pivot, they are swapped to the left side of it**. In **particular terms, in the partition(), we set index i = low – 1 for storing small elements and iterate j from low to high – 1. If arr[j] <= pivot, we increment i and perform the swap of arr[i] with arr[j]**. Finally, **we put the pivot in its proper place by making a swap of arr[i+1] and arr[j], getting pi = i + 1 as a result for the pivot's position**. Then, **with the quickSort(), we sort the left and right parts (low to pi – 1 and pi + 1 to high) recursively**. Thus, **having used the quickSort(dataset, 0, size – 1) in the main() function, we sort the array and print the 100 values in non-decreasing order with the header "—-Sorted array by Quick Sort—-"**, making the **Quick Sort algorithm have a O(n log n) time complexity**.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes it can by the use of **shellSort**. Let's look at my demonstration at the **hybrid quickSort Algorithm (shellSort Algorithm modifications)**

```cpp
#include <iostream>
#include <random>
#include "sortingAlgo2SupplPrb1.h"

const int size = 100;

int main() {
    int dataset[size];

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> dis(1, 100);

    for (int i = 0; i < size; i++) {
        dataset[i] = dis(gen);
    }


    std::cout << "\n----Sorted array by Quick Sort + Shell Sort----\n";
    quickSort(dataset, 0, size - 1);
    displayArray(dataset, size);


    return 0;
}
```

mainSupplPrb1.cpp code

```cpp
#ifndef SORTINGALGO2SUPPLPRB1_H
#define SORTINGALGO2SUPPLPRB1_H
#include <iostream>


template <typename T>
void displayArray(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

template <typename T>
void shellSort(T arr[], int size) {
    /* SHELL SORT
    shellSort(array, size)
        for intervaal i <= size /2n down to 1
            for each interval *i* in array
                sort all the elements at interrval *i*
    end shellSort
    */
    for (int gap = size / 2 ; gap >= 1; gap = gap / 2) {
        for (int i = gap; i < size; i++) {
            T temp = arr[i];
            int pos = i;
            while (pos >= gap && arr[pos - gap] > temp) {
                arr[pos] = arr[pos - gap];
                pos = pos - gap;
            }
            arr[pos] = temp;
        }
    }
}
```

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high]; // pivot
    int i = (low - 1);   // Index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++; // increment index of smaller element
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

template <typename T>
void quickSort(T arr[], int low, int high) {

    if (low < high) {
        // pi is partitioning index, arr[pi] is now at right place
        int pi = partition(arr, low, high);

        // Recursively sort elements before and after partition
        shellSort(arr, pi - 1);
        shellSort(arr, high);
    }
}
#endif // SORTINGALGO2SUPPL_H
```

sortingAlgo2SupplPrb1.h code

```
----Sorted array by Quick Sort + Shell Sort----
1 1 4 5 5 6 7 8 10 10 15 16 18 18 18 19 19 23 23 23 24 24 24 27 29 29 30 31 32 32 35 38 38 39 39 40 40 41 42 42 44
46 47 50 51 51 52 52 53 57 58 58 58 59 60 60 61 63 63 66 66 66 67 68 68 69 70 71 71 71 73 74 74 76 77 77 77 78 78 7
9 80 82 85 85 88 90 90 91 92 92 93 94 94 96 96 98 100 100 100 100
```

Console Output

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

I think in this array, the fastest sorting algorithm would be the **quick sort** because it **gives you the simple in-place partitioning loop of Quick Sort that is perfect for small datasets like this and avoids memory-management costs unlike the Merge Sort algorithm that has the literal long "Divide and Conquer" approach in the algorithm that has high memory costs**. Merge Sort and Quick Sort has the O(N log N) both for their time complexity because of their **"Divide and Conquer-based" Strategies which they assign the merging of the split subarrays (O(n)) and divide the array size into halves(log N) in Merge Sort**. In **Quick Sort, partitioning the subarrays around the chosen pivots involves the scanning and swapping of the elements across the range of the N elements (O(n)) and divide the array size into halves(log N)**.

CONCLUSION/LEARNINGS/ ANALYSIS	**Through this hands-on activity, I concluded that choosing between implementing Shell Sort, Merge Sort, and Quick Sort algorithms is dependent on the size of the data set, memory size allocation, and the design of the algorithm itself. I discovered that Shell Sort arranges data in a specific manner by initially setting a gap of size / 2 then dividing it by 2 (gap = gap / 2) and using insertion sort algorithm on gap intervals to minimize the distance between any two unsorted elements until a non-decreasing order is reached. Moreover, I found out that Merge Sort utilized recursion to break down an array into sub-arrays by setting a middle = (left + right) / 2 and merged them using a temporary array helper function merge(), which guarantees O(N logN) time complexity since merging split sub-arrays takes O(N) time and space complexity while dividing the array by halves (logN). In contrast, Quick Sort achieves O(N log N) time complexity using a "Lomuto partitioning scheme" that sets the last element as a pivot (arr[high]), swaps it to the correct position in the array (index pi = i + 1), and recursively sorts the sub-lists on either side of the pivot. By analyzing the performance characteristics of these algorithms, I discovered that Quick Sort is faster in smaller data sets because the in-place simple partition looping that it employs does not require the additional memory-management overhead required by the Merge Sort Algorithm while at the same time being flexible enough to implement hybrid implementations that sort partitioned sub-lists using Shell Sort in the Problem 1. In**

	my improvement that needs to be done, I want to improve my argument creation skills because it is needed for any sorting algorithms because there are times that I can't debug the code itself or the output didn't go as it should be so I need to improve this skill to apply this to the future lectures coding procedures efficiently.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **NONE**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **NONE**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **NONE**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.